

A Homomorphic Operator Layer for arithmetic that generalizes

Kronecker Embeddings V2 · individual submission · Kunal Sinha

Abstract

We consider embeddings that store mathematical structure — so that the vector of 9 combined with the vector of 9 becomes the vector of 18, and of 81. We show that *storing* arithmetic is the easy half: a generic spectral (Fourier) number code turns addition and multiplication into a single fixed operation (elementwise complex product) that is exact and needs zero parameters. The difficult half is making a *learned model* use that structure and generalize to numbers outside its training range — a normal network fed the correct structured features memorizes the training range and fails. Our contribution is the **Homomorphic Operator Layer (HOL)**: by making the exact operation an architectural primitive and learning only the mapping from a natural-language query to *which* operation, a tiny model answers arithmetic queries correctly on magnitudes it never saw (99.7%, 3 seeds), where an MLP on identical features scores near zero. The generalization lives in the operator, not the representation.

1. Spectral number codes

A number n is represented by a bank of complex phasors:

- additive code $a(n) = [e^{i \omega_k n}]_k$ — for $+$, $-$
- logarithmic code $m(n) = [e^{i v_k \ln n}]_k$ — for $\times$, $\div$

Both are deterministic and invertible: an integer is recovered by a matched filter that scores candidate values against the code's phases. Two homomorphisms follow directly from the phase, because shifting a number rotates every phasor and a logarithm turns products into sums:

$$a(x + y) = a(x) \odot a(y) \qquad m(x \cdot y) = m(x) \odot m(y)$$

where $\odot$ is the elementwise complex product. Subtraction and division use the conjugate. The operation is therefore a fixed, parameter-free **bind**, and the "embedding of 9 combined with the embedding of 9" literally decodes to 18 (additive band) and 81 (log band).

Exactness. With no trainable parameters, addition and subtraction decode exactly across the tested range ($a, b < 4000$), multiplication and division decode exactly across the tested product range, and chained additions remain exact to at least 32 terms with no drift.

2. The problem: structure that a model won't use

Exact storage does not imply a model can compute with it and extrapolate. If the spectral features of a and b are handed to a standard multilayer network that must produce $a + b$, the network fits the training range but collapses on larger, unseen magnitudes. This is the core difficulty: the structure is present in the representation, but a generic learned readout does not inherit it.

3. The Homomorphic Operator Layer

The layer separates what should be learned from what is fixed:

1. **Encode.** Numbers become spectral codes; the words of the query become ordinary learned token embeddings.
2. **Gate.** A small learned gate maps the *phrasing* of the query ("plus"/"add"/"sum" → `+`) to one of the operations. This is the only trained component, and it generalizes across phrasings.
3. **Bind and decode.** The selected exact bind combines the two number-codes and a fixed decoder reads out the integer. This step has no parameters and is valid at any magnitude.

The network thus learns only *what the query asks for*; it never learns arithmetic, which is fixed and exact. That is precisely what converts "memorize a range" into "compute for any magnitude."

4. Experiment and result

Task. Templated natural-language arithmetic queries "`<a> <word> <b>`", with several phrasings per operation across `+`, `-`, `×`. Numbers are drawn from `0..119`. **Training** uses only pairs whose operands are below 80; **testing** uses pairs whose larger operand is at least 80 — magnitudes absent from training. Reported as the mean over three seeds.

model	train (in-range)	unseen magnitude
Homomorphic Operator Layer	0.99	0.997
MLP on the same spectral features	~0.01	~0.001

The operator layer generalizes almost perfectly to unseen magnitudes; the learned baseline on identical inputs does not. The result isolates the cause of arithmetic generalization: it is architectural. Making the homomorphic operation a primitive, rather than hoping a network learns it from structured features, is what yields a model whose arithmetic holds outside the training distribution.

5. Discussion

The number code is appended alongside a model's existing word or byte dimensions, matching the intended "keep the existing dimensions for language, add mathematics in new ones" design. Two directions extend the result: (i) making the decode step differentiable end-to-end so the gate can be trained purely from answer supervision; and (ii) handling magnitude beyond a fixed decode window, where the log code's monotonic phase provides a natural, order-preserving signal. Everything reported here is deterministic, self-contained, and reproducible on CPU in about a minute.

All figures are produced by the accompanying code (`experiments/exactness.py` , `experiments/operator_layer.py`); the interactive companion runs the same computation live in the browser.